Mahidol University
Faculty of Information
and Communication Technology

OS Security and Access Control

Ittipon (Art) Rassameeroj

About Me

- https://ittiponr.github.io
- Education
  - Ph.D. in Computer Science at University of California, Davis, USA
  - M.S. and B.S. in Computer Science at Mahidol University
- Interests
  - Cyber security
  - Computer systems
  - Operating/distributed systems, parallel/cloud/high-performance computing
- Current Teaching
  - Computer and network security (ICT, DST, CY)
  - Cloud security (ICT, CY)
  - Parallel and High Performance Computing (ICT)
  - Introduction to Linux, Shells, and Commands (ICT)

Agenda

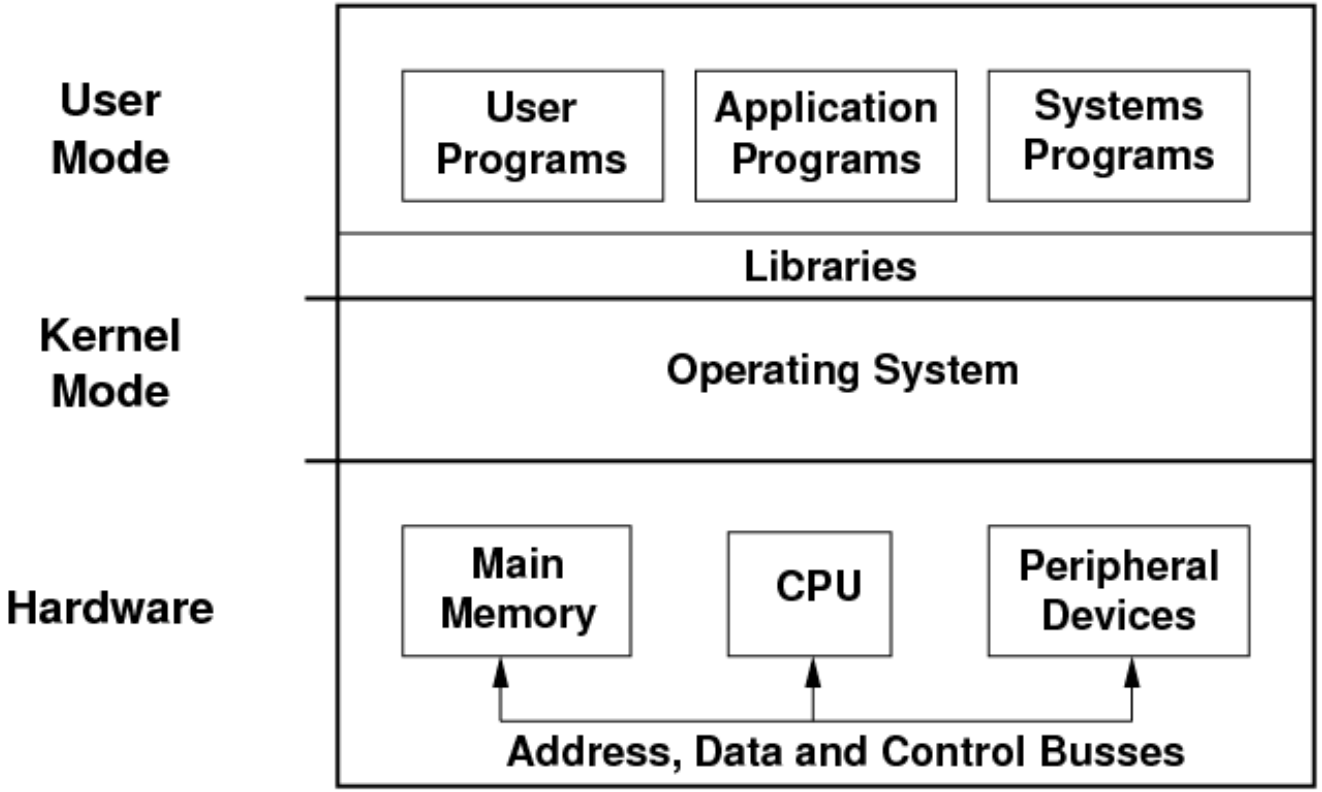
OS Security (Chapter 12)

- Introduction to OS
- OS security functions
- Memory protection

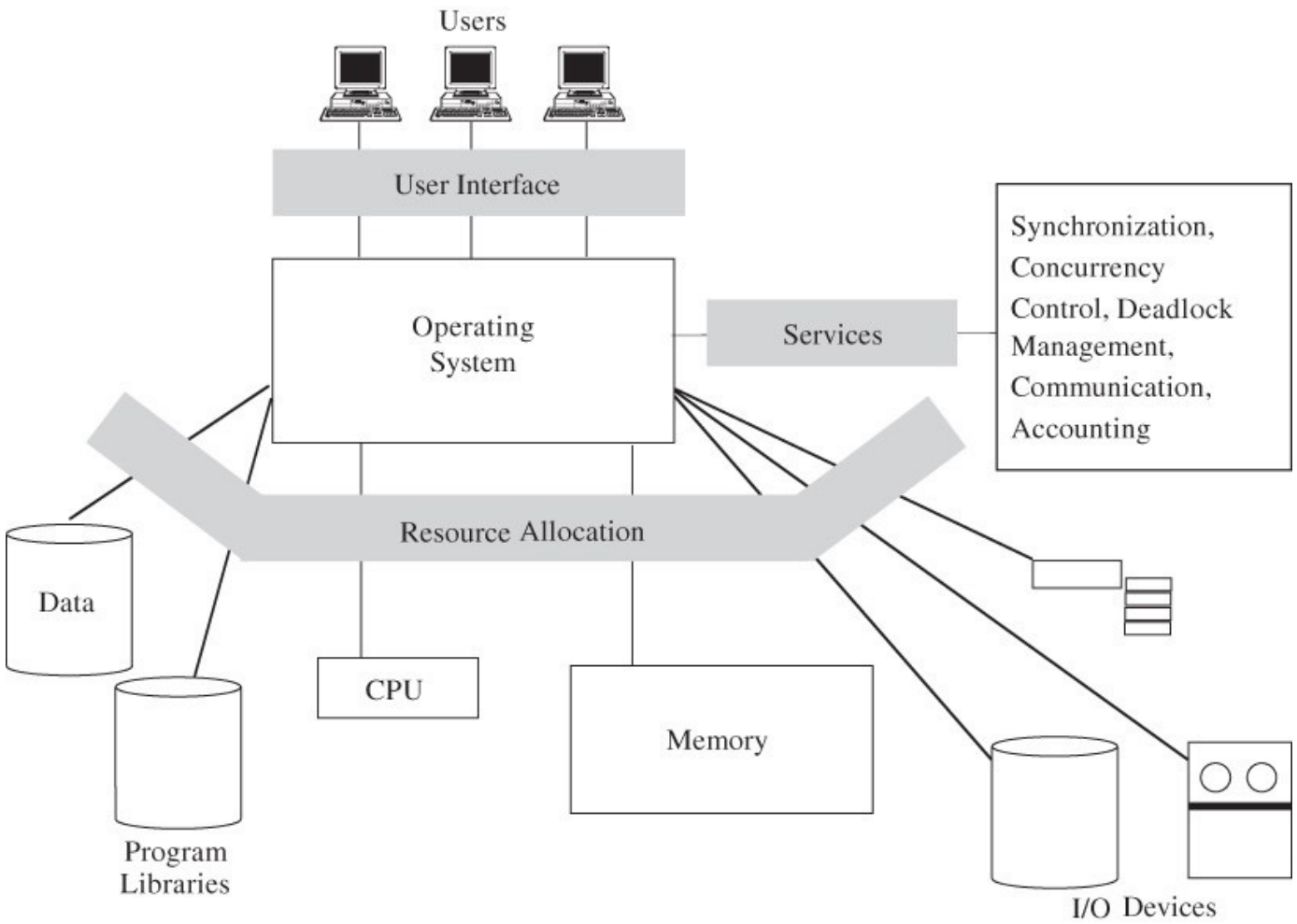
Access control (Chapter 4)

- Access control principles, guidelines, and elements
- Access control mechanisms
- Access Control in Unix
- Access control policies

OS Structure



OS Structure



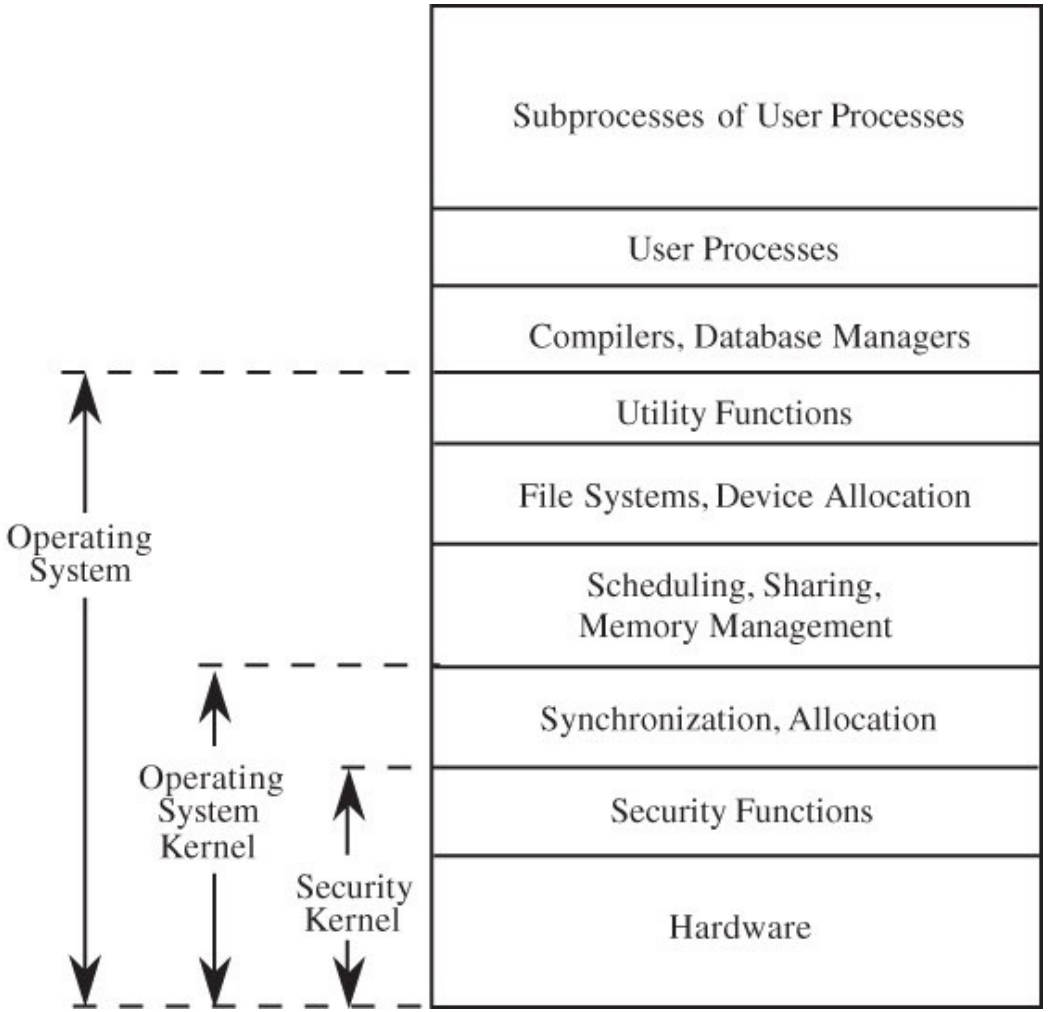
Security in OS

- Goals of OS
  - Controlling share access
  - Implementing an interface to allow that access
- Basic OS security: separation and sharing
  - Keeping one user's objects secure from being interfered by other users
  - However, OS must provide a way to do sharing as well.
- OS is the fundamental controller of all system resources which makes it a primary target of attack.
- OS is the first line of defense against all sorts of unwanted behaviors.

OS Security Functions

- Identity and credential management
- Performs identification and authentication of people and remote operations
- Access control
- Protects one user from another, ensures that critical areas of memory or storage are not overwritten by unauthorized processes.
- OS provides different levels of protection for different objects/resources.
- Information flow
- Audit (logging) and integrity protection (e.g., memory protection)

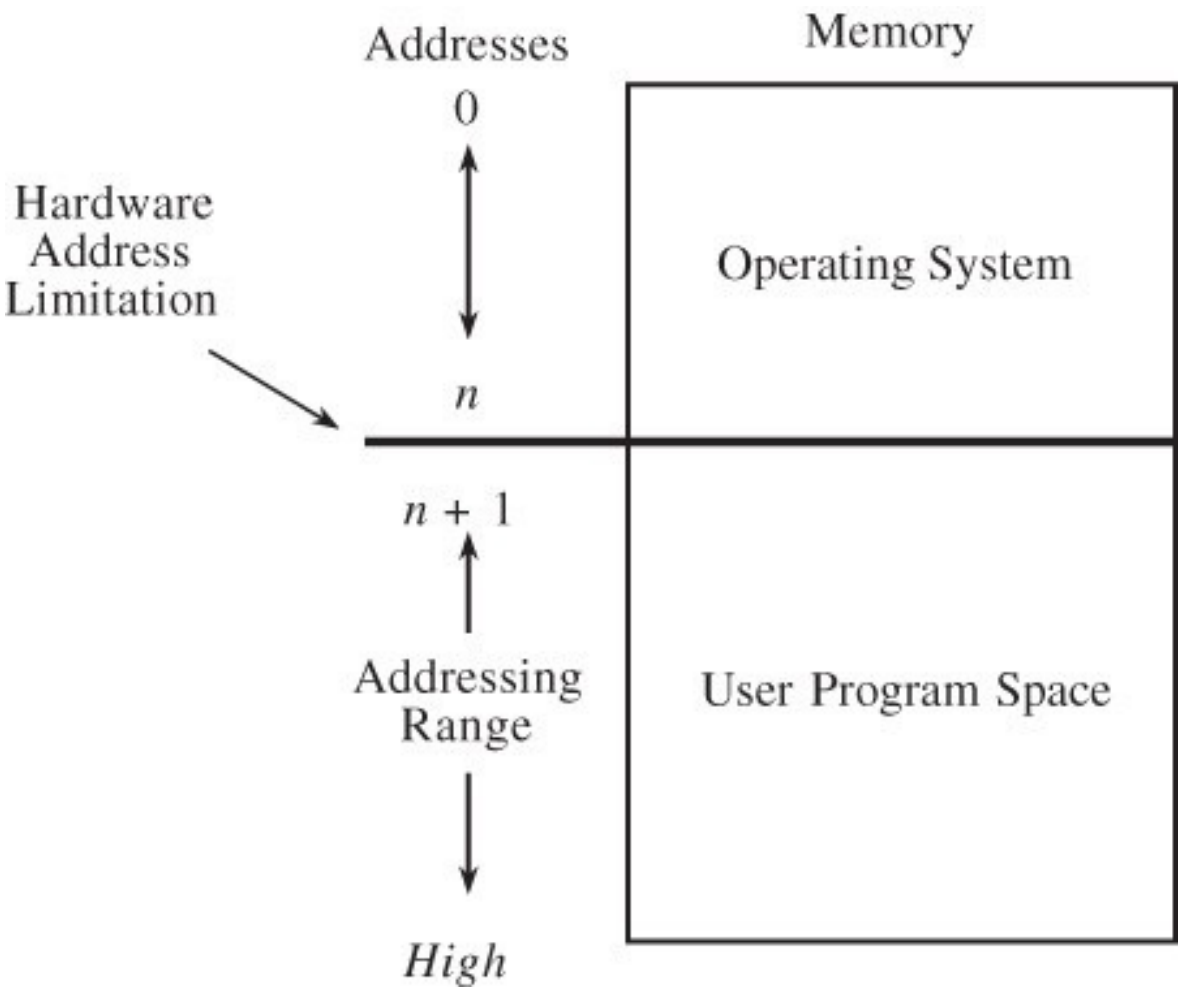
OS Layers


Subprocesses of User Processes
User Processes
Compilers, Database Managers
Utility Functions
File Systems, Device Allocation
Scheduling, Sharing,
Memory Management
Synchronization, Allocation
Security Functions
Hardware
Operating
System
Operating
System
Kernel
Security
Kernel

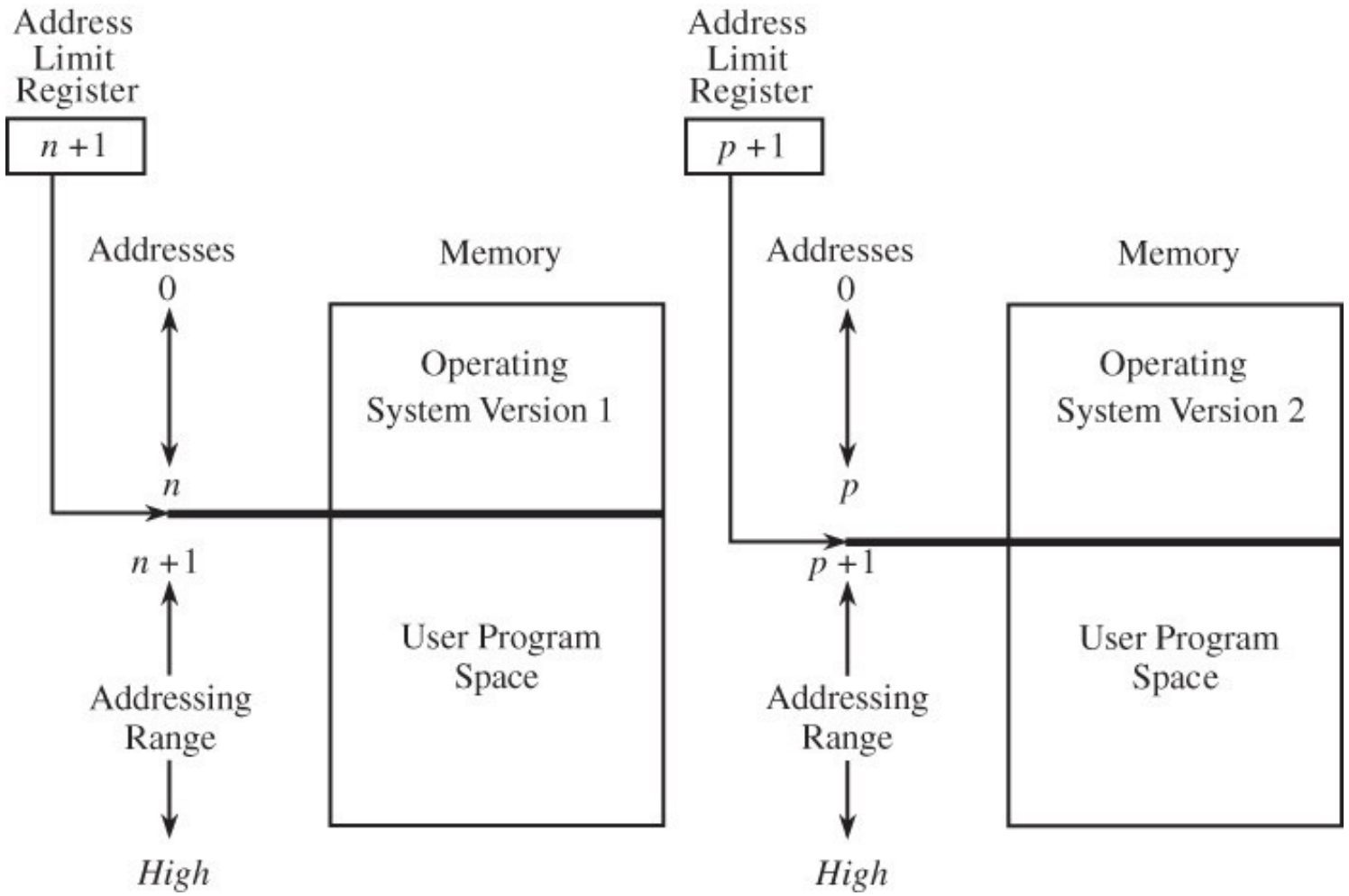
Hardware Protection of Memory

- OS protect each program's memory from being affected by other programs
- Protection mechanisms
  - Fence protection
  - Fence registers and base/bounds registers
  - Tagged Architecture
  - Virtual Memory: segmentation and paging

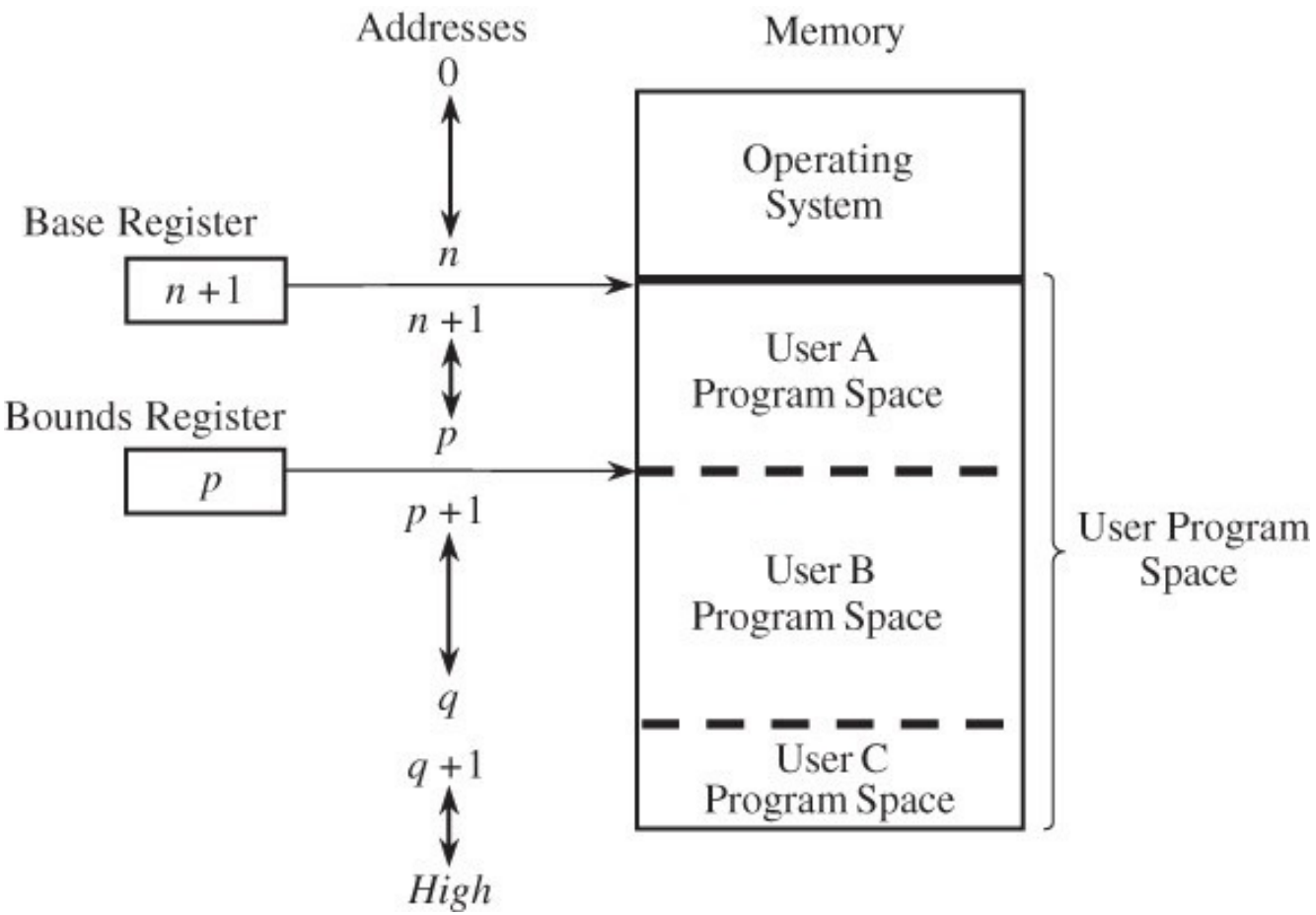
Fence Protection



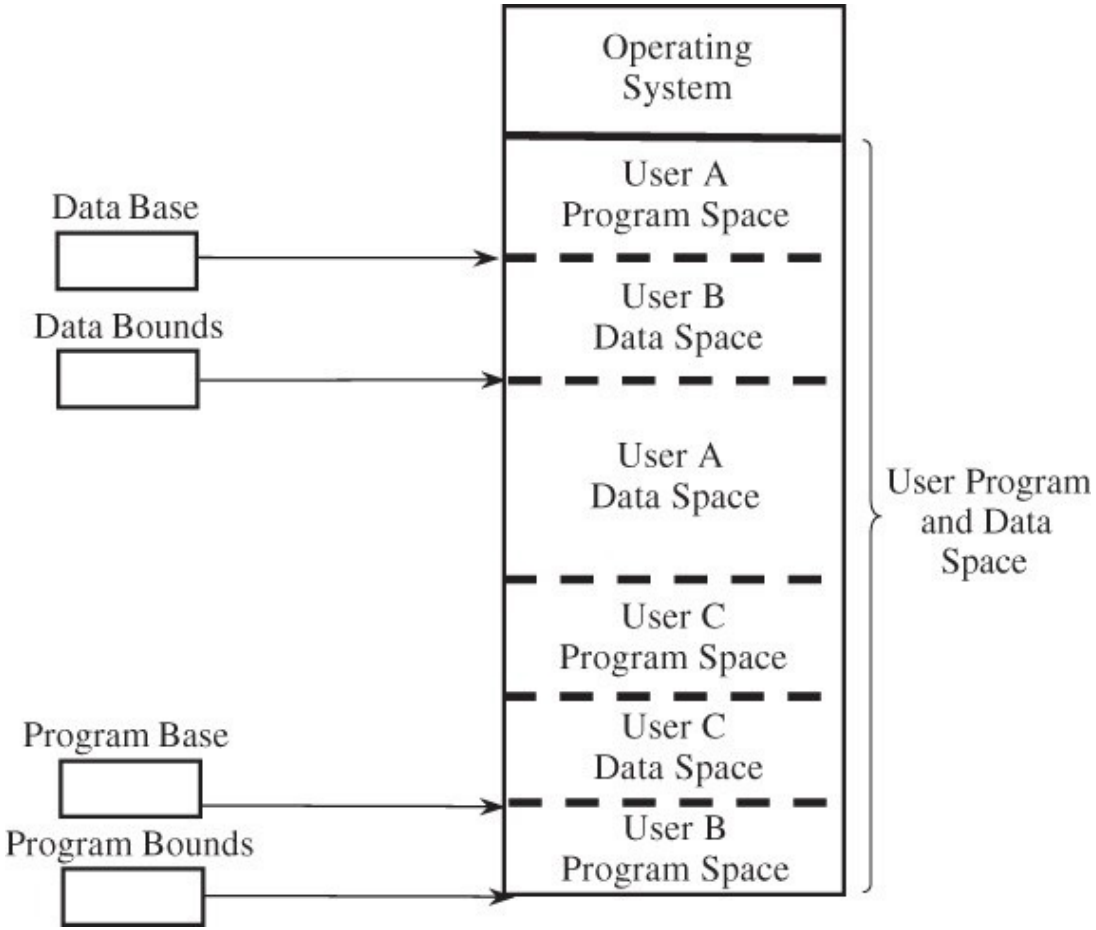
Fence Register



Base and Bound Registers



Base and Bound Registers



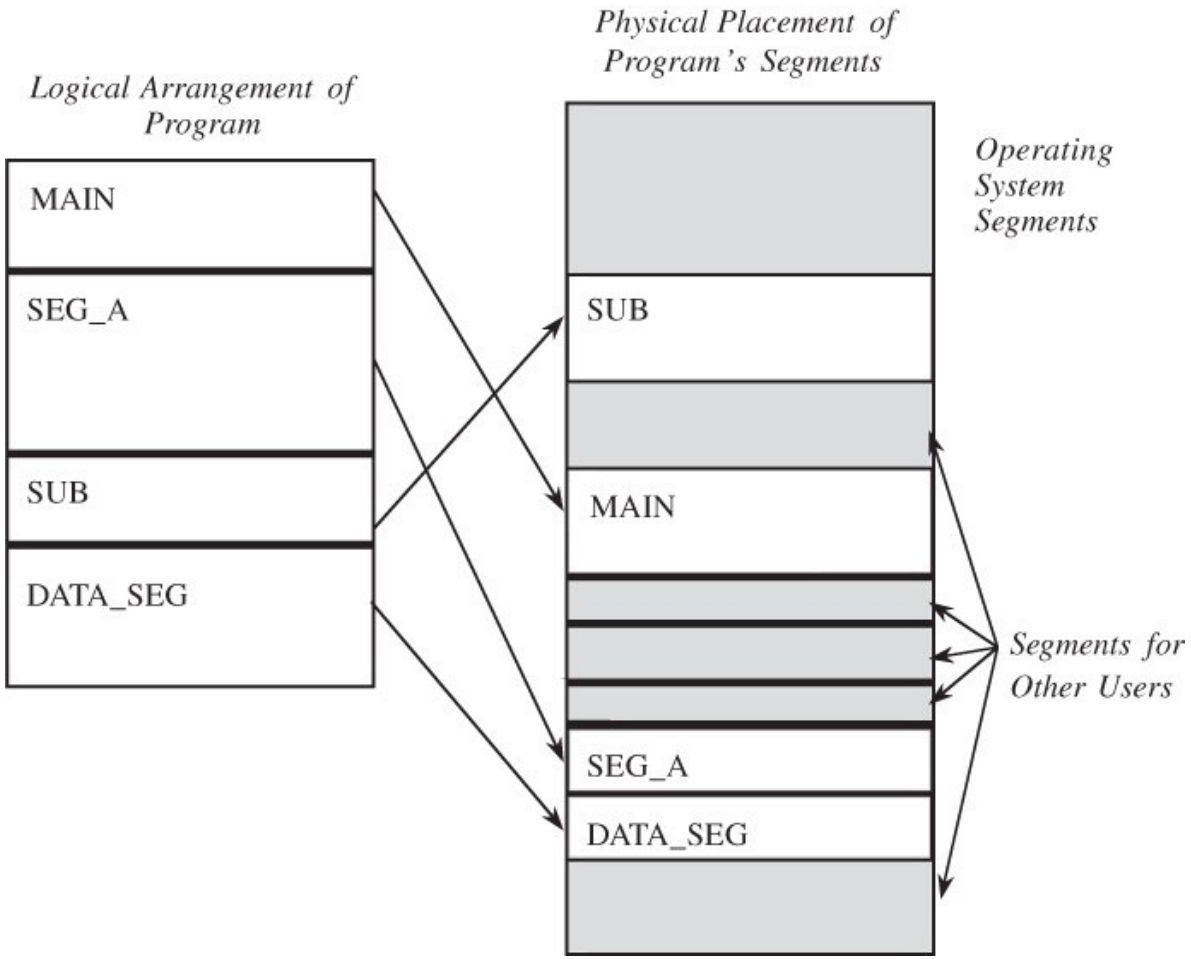
Tagged Architecture

Tag	Memory Word
R	0001
RW	0137
R	0099
X	
X	
X	
X	
X	
X	
R	4091
RW	0002

Code: R = Read-only RW = Read/Write
X = Execute-only

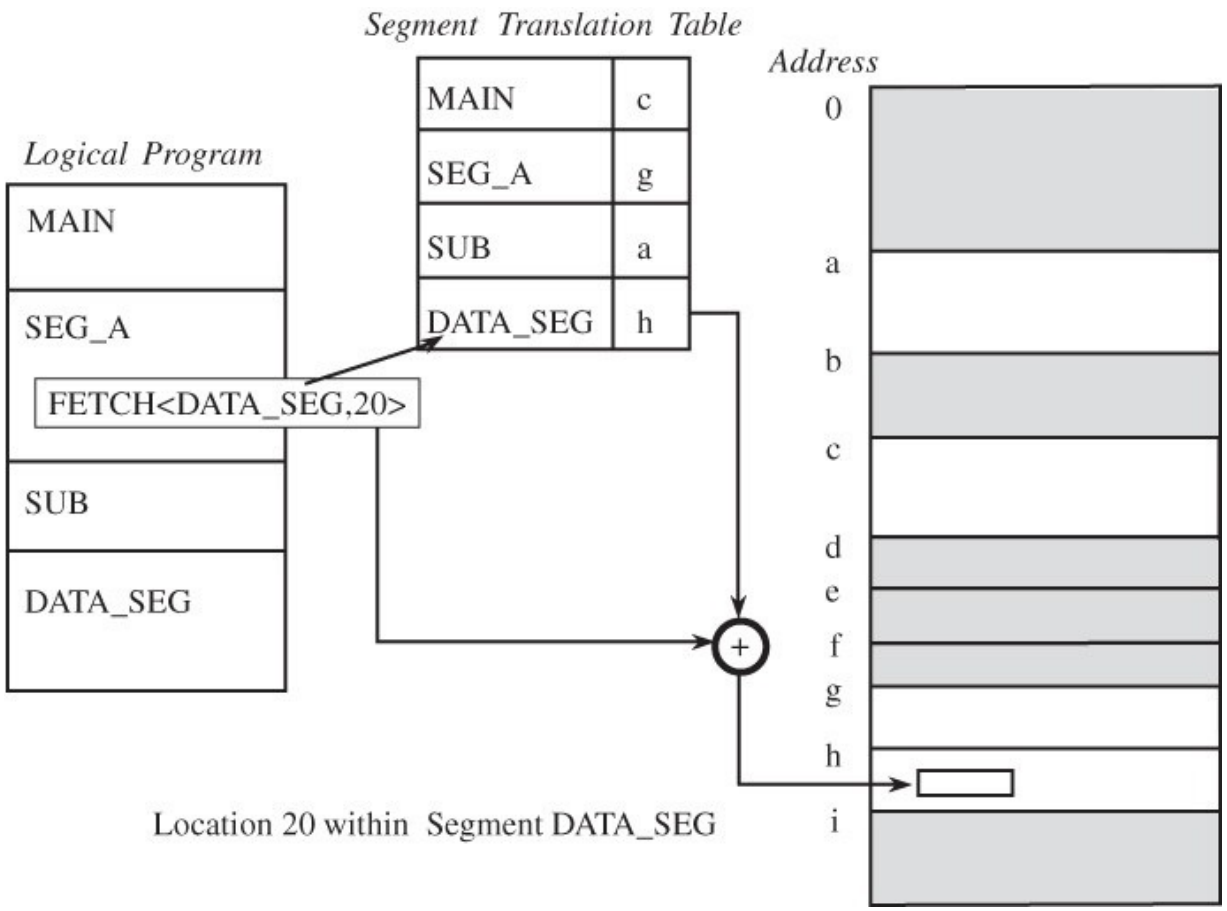
- Every word of machine memory has one or more extra bits to identify the access rights to that word.
- These access bits can be set only by privileged (OS) instructions.
- The bits are tested every time an instruction accesses that location.

Virtual Memory Segmentation

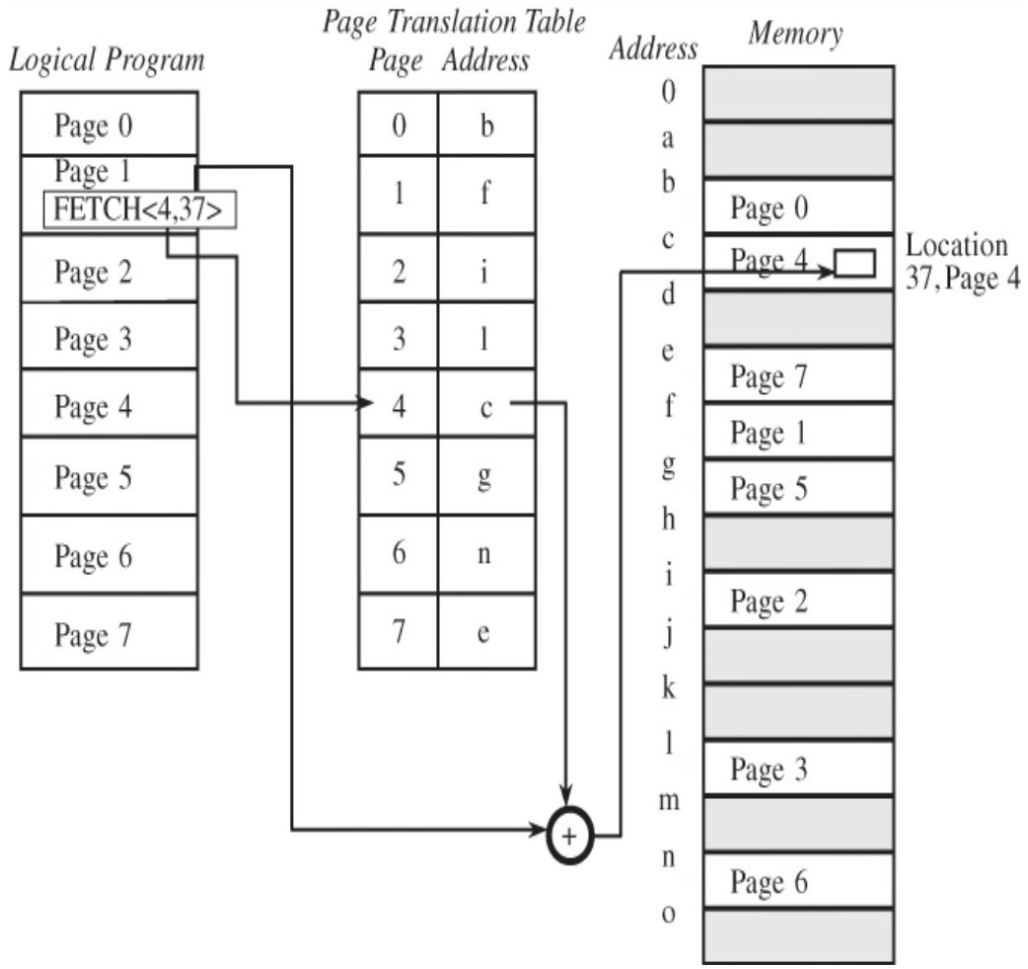


Segmentation allows hardware-supported controlled access to different memory sections in different access modes

Virtual Memory: Segment Address Translation



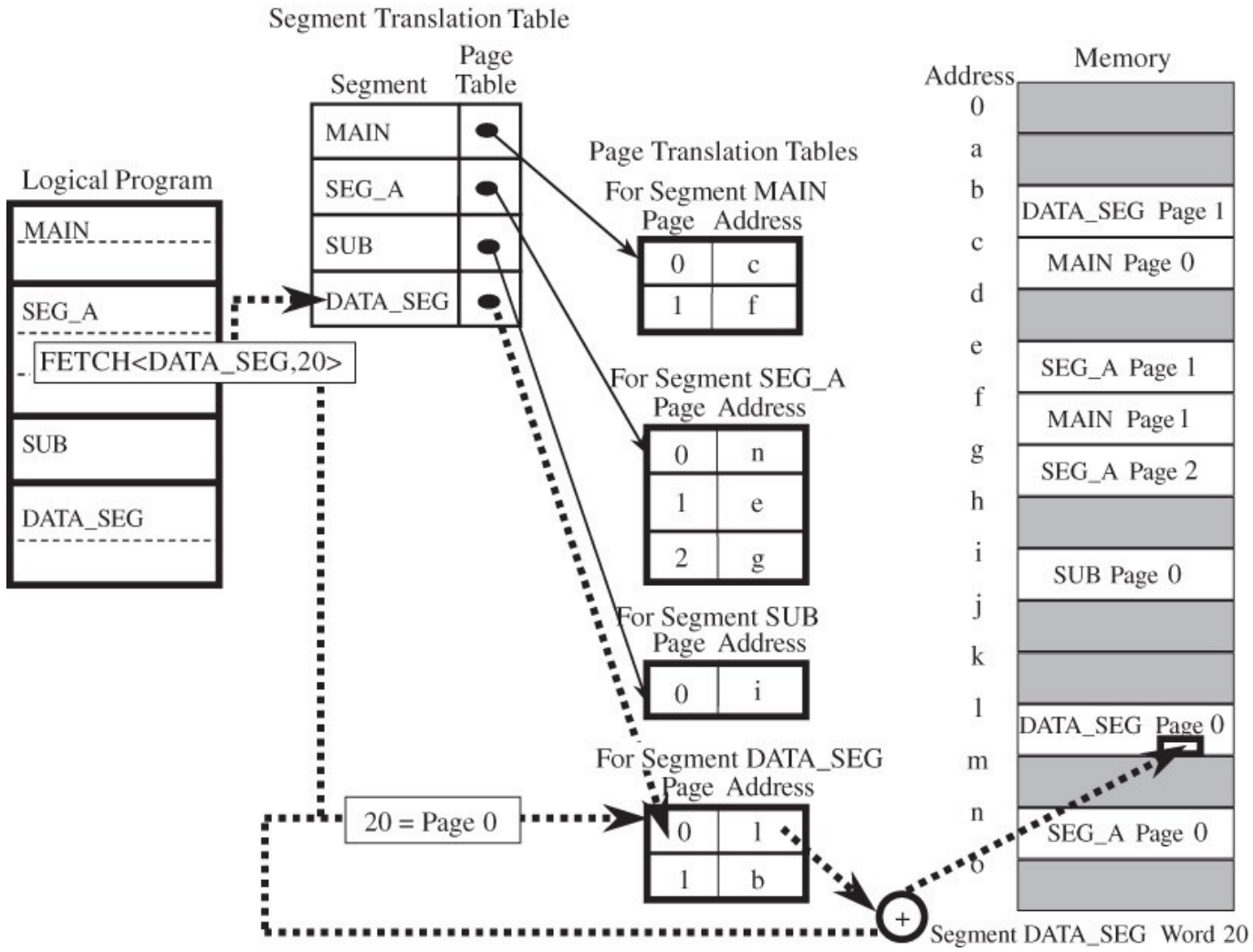
Virtual Memory: Paging



Paging gives efficient memory management by

- A program is divided into pages.
- Memory is divided into page frames.
- Page size is usually chosen to be a power of 2: 2 KB, 8 KB, 16 KB.

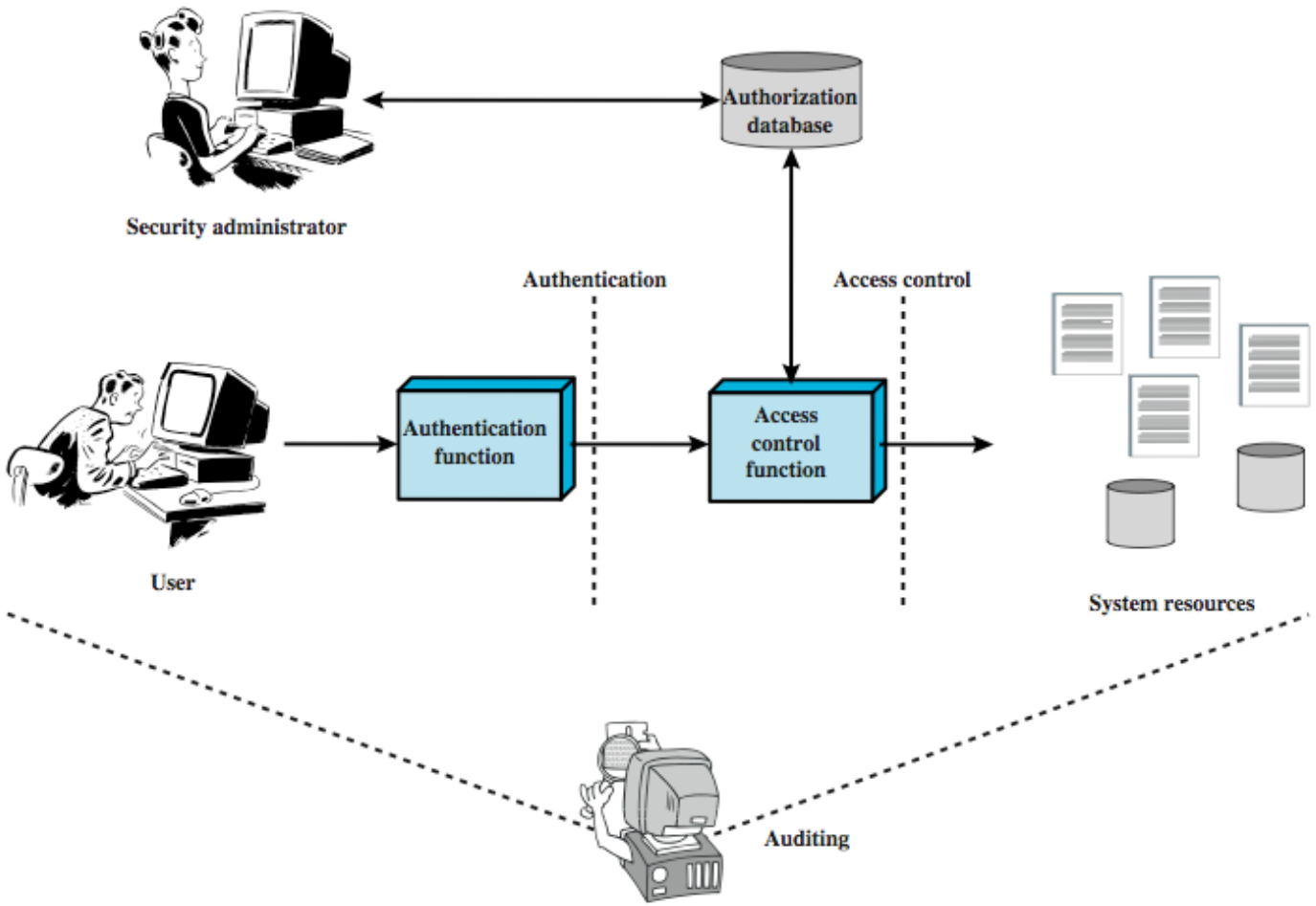
Virtual Memory: Page & Segmentation Combination



Access Control

- The prevention of unauthorized use of a resource, including the prevention of use of a resource in an unauthorized manner.
- Central element of computer security
- Access control implements a security policy that specifies who may have access to what resource in what access type
- Access control involves:
  - Authentication: verification of user entity and validity
  - Authorization: permission to a user entity to access system resource
  - Audit: review & examination of system records and activities

Access Control Principles



Access Control Guideline

- Check every access: Access is not granted forever—can be suspended or revoked.
- Enforce least privilege: Give a subject access to the smallest number of objects necessary to perform the subject's task.
- Verify acceptable use: Verify if the requested kind of access is acceptable; for example, read is OK, but write + execute are not.

Access Control Elements

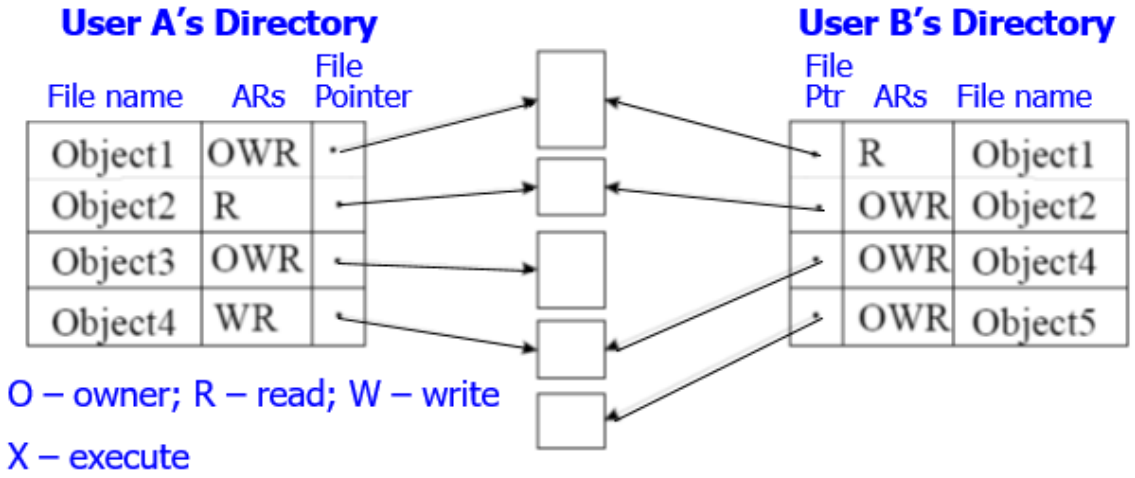
- Subject - entity that can access objects
  - Users
  - Processes/threads representing user or application
- Object - access controlled resource
  - Such as files, directories, records, programs, hardware devices, etc.
  - Types of objects depend on environment.
- Access right - way in which subject accesses an object
  - Such as read, write, execute, delete, create, search, transfer, etc.

Basic Access Control Mechanism

- Directory
- Access Control List
- Access Control Matrix
- Capability

Directory Mechanism

- Unique object owner and users are a subject.
- Owner controls access rights [rwx bits] by assigning or revoking them.
- Each user has access rights directory.



Access Control List (ACL)

A list attached to an object

- Specifying access rights for each subject accessing the object
- Subject could be individuals, groups, or others.

```
Object1: {{A: OWR}, {B: R}, {C: R}, {D: R}}
Object2: {{A: R}, {B: OWR}, {C: R}, {E: R}}
Object3: {{A: OWR}}
Object4: {{B: OWR}, {*: WR}}
Object5: {{B: OWR}, {E: R}}
```

Access Control Matrix

- Directory – subject's list of ARs for objects accessible by the subject
- Access control list – object's list of ARs for subjects that can access the object.
- Access control matrix – a sparse matrix (a table)
- Rows for subjects and columns for objects
- Cell(*i, j*) : subjects *i*'s access rights to object *j*

	Object 1	Object 2	Object 3	Object 4	Object 5
Subject A	OWR	R	OWR	WR	-
Subject B	R	OWR	-	OWR	OWR
Subject C	R	R	-	WR	-
Subject D	R	-	-	WR	-
Subject E	-	R	-	WR	R

Capabilities for Access Control

- Subjects access objects only via capabilities
- Capability
  - A kind of token/ticket given to subject certain rights for an object
  - To see (kind of access) a movie (object), an audience (subject) must have a ticket (capability)
- Capability to transfer access rights
  - Allows subject to pass copies of its capabilities to other subjects
  - S_1 can copy its capability to access O_1 and transfer it to S_2. If S_1 omits 'transfer' rights for O_1 in capability passed to S_2, S_2 can't transfer these rights to any other subject.

Access Control Policies

- Tell **what types of access** permitted under **what circumstances** and **by whom**.
- Discretionary (DAC): control access based on requestor and volition of resource owner
- Mandatory (MAC): control access based on security labels and clearances specified centrally
- Role-based (RBAC): control access based on roles and rules (access allowed in given roles)
- Attribute-based (ABAC): control access based on attributes, resource, and environment

Discretionary Access Control (DAC)

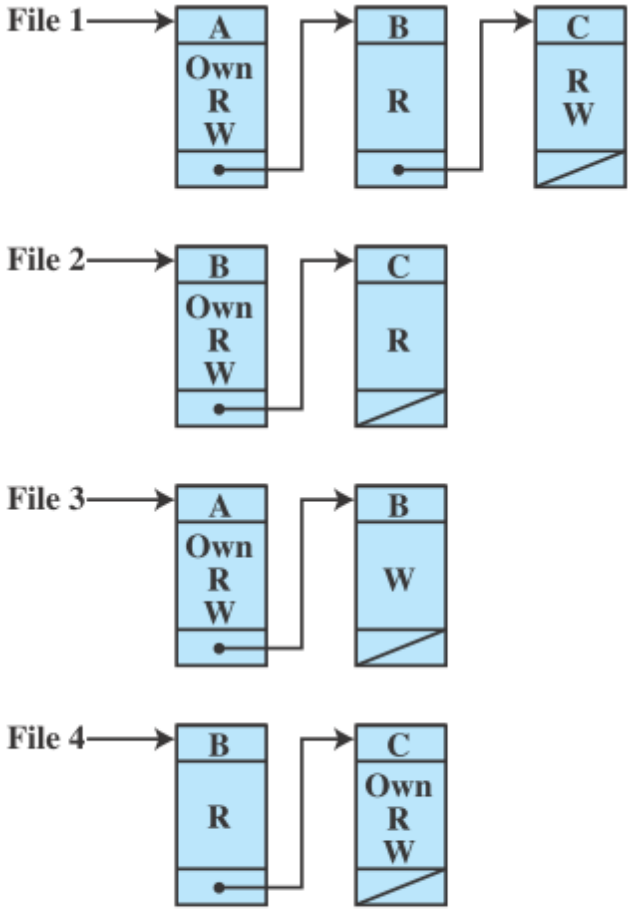
- It allows the owner, creator, or data custodian of an object to control and define access to that object.
- All objects have owners, and access control is based on the discretion or decision of the owner.
- Often provided using an access control matrix
- Access control matrix is often sparse which can be decomposed by either row or column.

Discretionary Access Control (DAC)

Decomposed by columns

SUBJECTS	OBJECTS: File 1	File 2	File 3	File 4
User A	Own Read Write		Own Read Write	
User B	Read	Own Read Write	Write	Read
User C	Read Write	Read		Own Read Write

(a) Access matrix



(b) Access control lists for files of part (a)

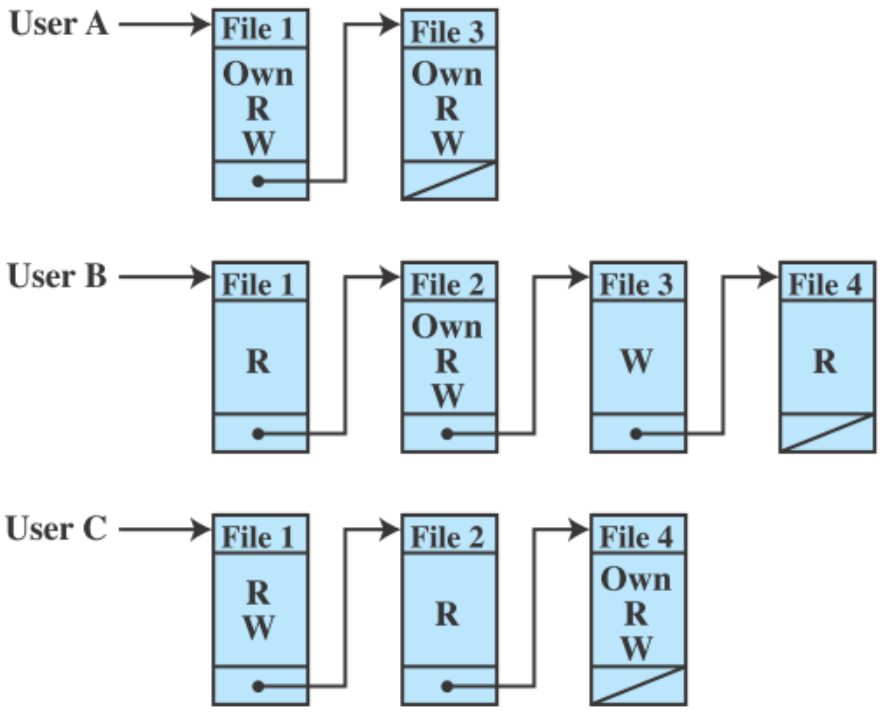
Result in ACL

Discretionary Access Control (DAC)

Decomposed by rows

SUBJECTS	OBJECTS: File 1	File 2	File 3	File 4
User A	Own Read Write		Own Read Write	
User B	Read	Own Read Write	Write	Read
User C	Read Write	Read		Own Read Write

(a) Access matrix



(c) Capability lists for files of part (a)

Result in CL

Discretionary Access Control (DAC)

- DAC in OS is more complicated than just accessing to the file.
- It contains other types of object such as process, device, etc.

		OBJECTS								
		subjects			files		processes		disk drives	
		S_1	S_2	S_3	F_1	F_2	P_1	P_2	D_1	D_2
SUBJECTS	S_1	control	owner	owner control	read *	read owner	wakeup	wakeup	seek	owner
	S_2		control		write *	execute			owner	seek *
	S_3			control		write	stop			

* - copy flag set

Figure 4.3 Extended Access Control Matrix

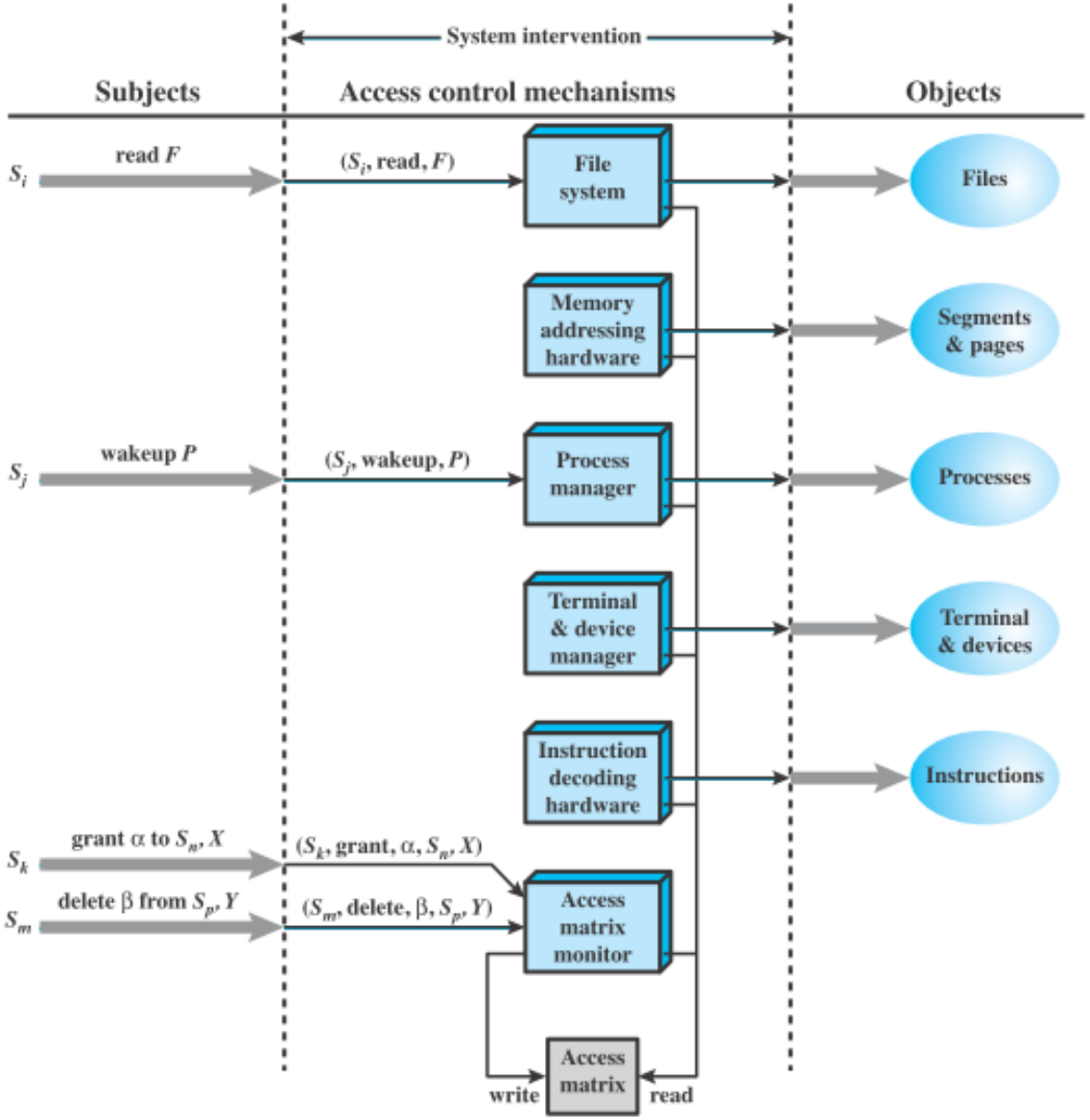
DAC

- It needs “access control function”
- Access attempt triggers 3 steps

  1) Subject *S* issues request of type α for object *X*.

  2) OS generates message (*S*, α, *X*) to the controller for *X*.

  3) Controller checks access matrix A to determine if α is in A[S, X].

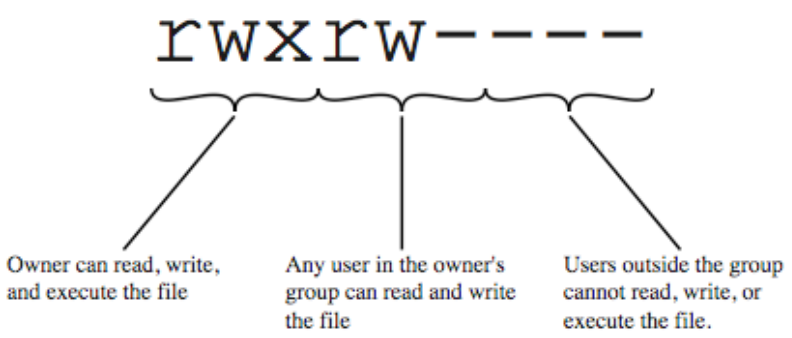


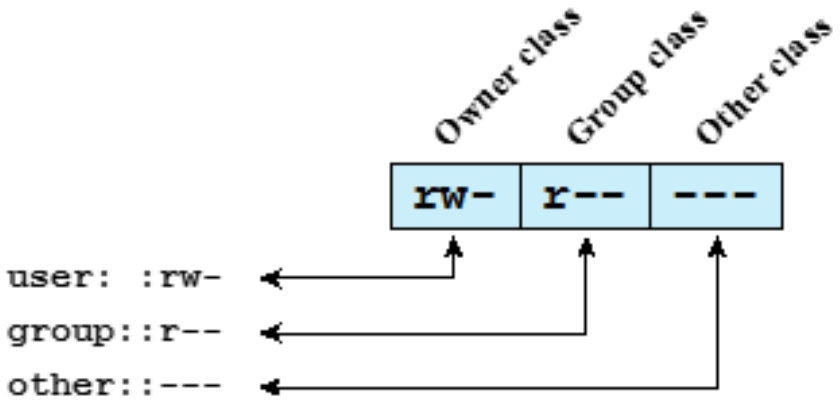
Access Control System Commands

Rule	Command (by S_0)	Authorization	Operation
R1	**transfer** $\left\{\begin{matrix}\alpha^* \\ \alpha\end{matrix}\right\}$ **to** S, X	"α^*" in $A[S_0, X]$	store $\left\{\begin{matrix}\alpha^* \\ \alpha\end{matrix}\right\}$ in $A[S, X]$
R2	**grant** $\left\{\begin{matrix}\alpha^* \\ \alpha\end{matrix}\right\}$ **to** S, X	'owner' in $A[S_0, X]$å	store $\left\{\begin{matrix}\alpha^* \\ \alpha\end{matrix}\right\}$ in $A[S, X]$
R3	**delete** α **from** S, X	'control' in $A[S_0, S]$ or 'owner' in $A[S_0, X]$	delete α from $A[S, X]$
R4	$w \leftarrow$ **read** S, X	'control' in $A[S_0, S]$ or 'owner' in $A[S_0, X]$	copy $A[S, X]$ into w
R5	**create object** X	None	add column for X to A; store 'owner' in $A[S_0, X]$
R6	**destroy object** X	'owner' in $A[S_0, X]$	delete column for X from A
R7	**create subject** S	none	add row for S to A; execute **create object** S; store 'control' in $A[S, S]$
R8	**destroy subject** S	'owner' in $A[S_0, S]$	delete row for S from A; execute **destroy object** S

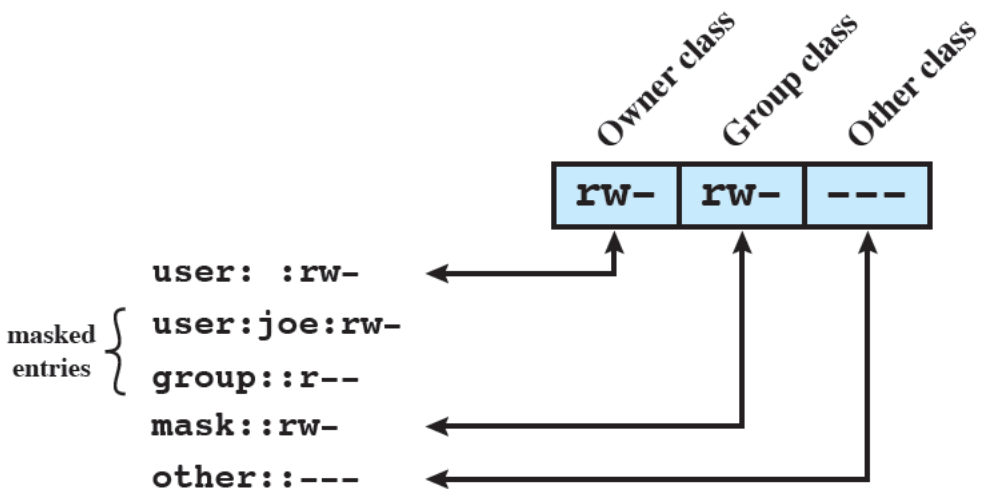
UNIX File Access Control

- Unique user identification number (user ID)
- Member of a primary group identified by a group ID
- 12 protection bits
  - 9 specify read, write, and execute permission for the owner of the file, members of the group and all other users
  - 2 specify SetUID, SetGID
  - 1 is the sticky bit (only owner can move, remove, delete a file)
- The owner ID, group ID, and protection bits are part of the file's inode.

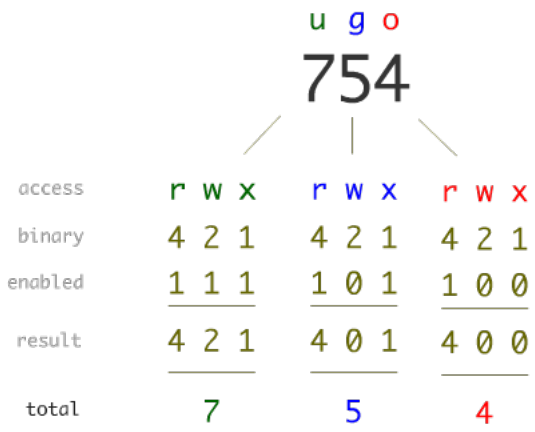




UNIX File Access Control



(b) Extended access control list



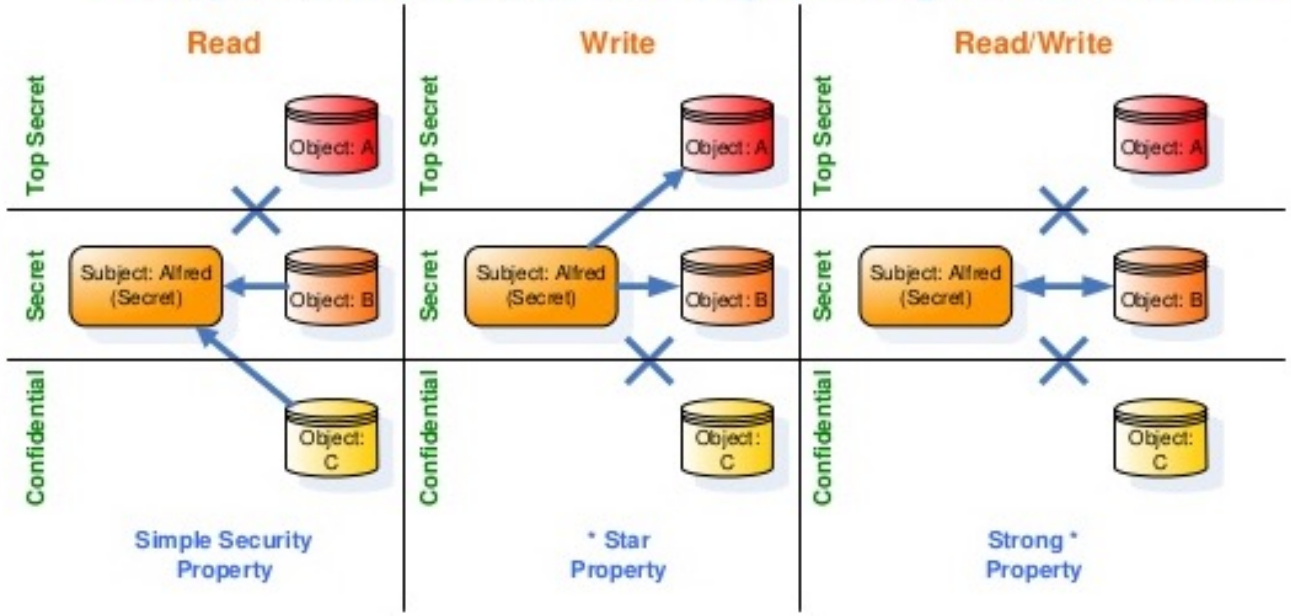
	4	2	1	
0	-	-	-	no permissions
1	-	-	x	only execute
2	-	w	-	only write
3	-	w	x	write and execute
4	r	-	-	only read
5	r	-	x	read and execute
6	r	w	-	read and write
7	r	w	x	read, write and execute

Mandatory Access Control (MAC)

- Aka. Multilevel Security
- MAC is a policy in which people don't have control over the authorization of people to information
- Access control is based on a policy, e.g., a security labeling system.
- It is used in environments where information classification and confidentiality is very important; such as in the military
- For example, in Bell-LaPadula Model, access is allowed if
  - subject clearance level >= object access class and
  - object categories subset-of subject categories
  - Basically, called No Read up and No Write down

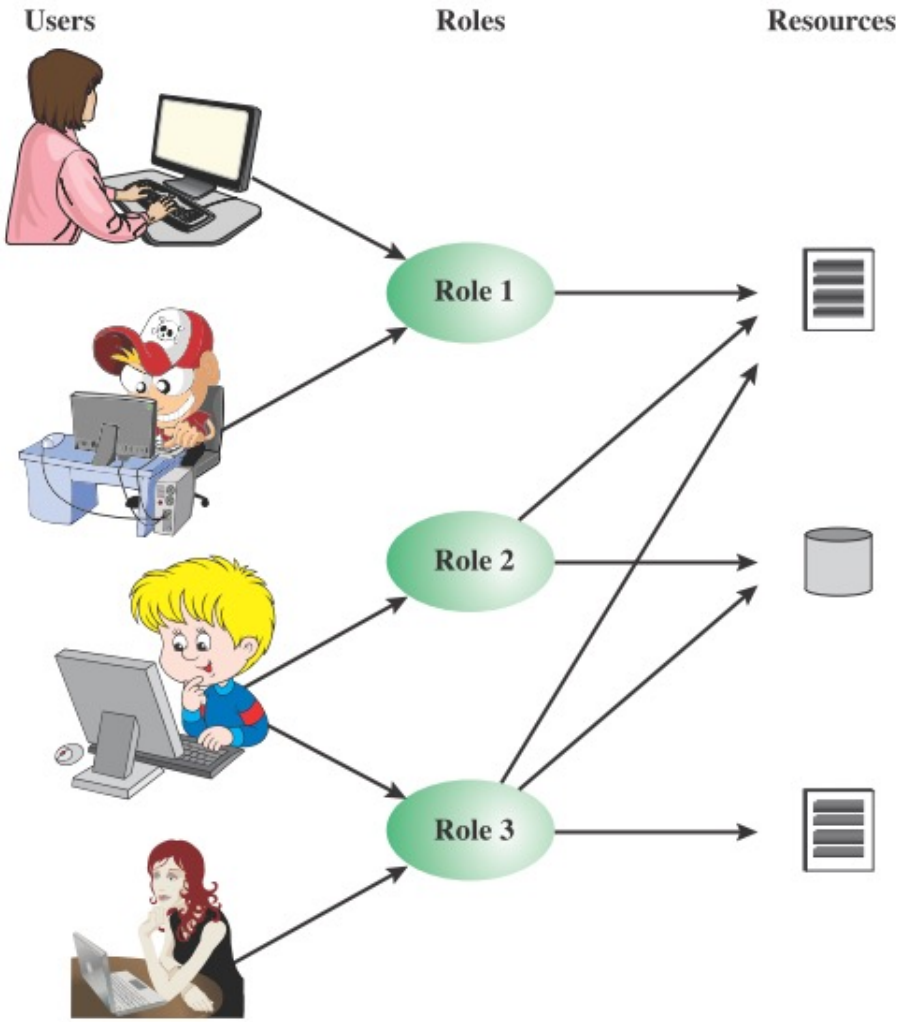
MAC Example: Bell-LaPadula Model

- Simple security property
  - Subject cannot read object of higher sensitivity.
- Star property (* property)
  - Subject cannot write to object of lower sensitivity.
- Strong Star property (Strong * property)
  - Subject cannot read/write to object of higher/lower sensitivity.



Role-Based Access Control (RBAC)

- Based on roles which are assigned to the users
- Define a role as a job function in organization.
- Assign access rights to roles instead of individual users.
- Roles assigned may be static or dynamic.
- Assignment is many-to-many.
- Task-based access control (TBAC) is similar to RBAC, but instead of being assigned to one or more roles, each user is assigned an array of tasks.
  - As an example, Microsoft Project uses TBAC. Each project has multiple tasks.



RBAC on Access Control Matrix

- Associate users to roles
- Associate roles to objects using DAC access control matrix
- Role should contain minimum set of access rights needed

ROLES \ OBJECTS	R_1	R_2	R_n	F_1	F_1	P_1	P_2	D_1	D_2
R_1	control	owner	owner control	read *	read owner	wakeup	wakeup	seek	owner
R_2		control		write *	execute			owner	seek *
• • •									
R_n			control		write	stop			

	R_1	R_2	• • •	R_n
U_1	X			
U_2	X			
U_3		X		X
U_4				X
U_5				X
U_6				X
• • •				
U_m	X			

Attribute-Based Access Control (ABAC)

- Define authorization according to properties of resource and subject.
- Flexibility and expressive
- A logical access control model
- Controls access to objects by evaluating rules against the attributes of subjects, objects, operations, and the environment relevant to a request.
- Capable of enforcing DAC, MAC, and RBAC concepts
- ABAC enables fine-grained access control.
- Allows unlimited number of attributes to be combined to satisfy any access control rule.

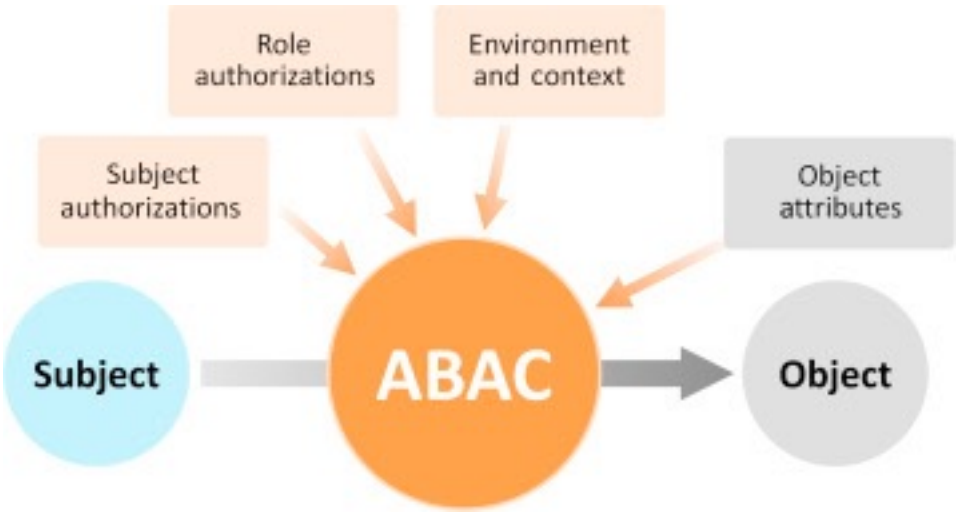


Figure: https://www.sciencedirect.com/topics/computer-science/attribute-based-access-control

Attribute-Based Access Control (ABAC)

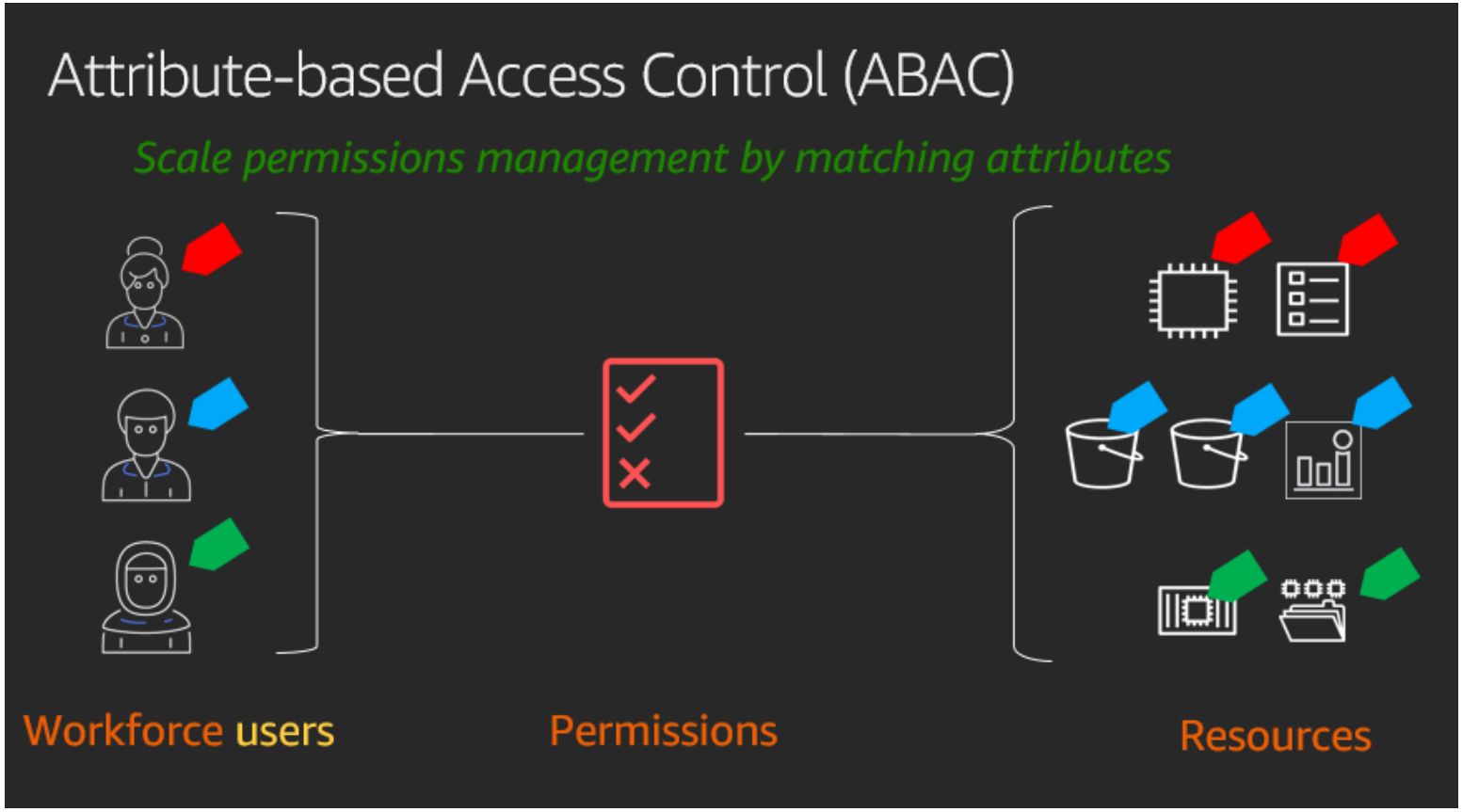


Figure: https://aws.amazon.com/blogs/security/tag/attribute-based-access-control/